

GraphRAG vs 普通 RAG 原理详解

项目概述

本项目实现了一个基于 **Ollama + Neo4j + Flask** 的知识图谱问答系统，支持两种检索模式：

- 1. **GraphRAG**：基于知识图谱的检索增强生成
- 2. **普通RAG**：基于向量数据库的检索增强生成

一、GraphRAG 工作原理

1.1 图分解（知识抽取）

GraphRAG 的核心是将非结构化文本转换为结构化的知识图谱。这个过程称为**图分解**。

1.1.1 三元组提取

系统使用 LLM（Qwen2.5）从文本中提取结构化的知识三元组：

```
def extract_triplets(text: str) -> list:
    """使用 Ollama 从文本中提取三元组"""
    prompt = f"""从以下文本中提取实体和关系，输出格式：
    subject|relation|object

    重点提取：人与人、人与公司、公司与地点、公司与类型等关系

    示例：
    苹果公司|行业|科技公司
    蒂姆·库克|担任职位|苹果公司CEO
    """
    response = llm.complete(prompt)
    return triplets
```

1.1.2 转换示例

输入文本：

王五是谷歌公司的CEO，他提倡人工智能技术。

输出三元组：

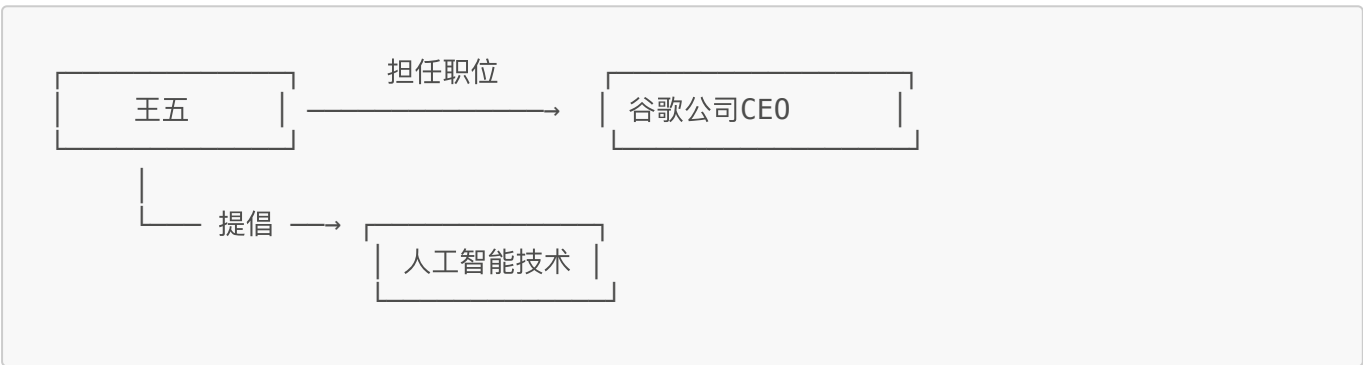
王五|担任职位|谷歌公司CEO
王五|提倡|人工智能技术

1.2 存储到 Neo4j

提取的三元组使用 **Cypher 查询语言** 存储到 Neo4j 图数据库：

```
for subject, relation, obj in triplets:
    neo4j_conn.query(
        "MERGE (a:Entity {name: $subject}) "
        "MERGE (b:Entity {name: $object}) "
        "MERGE (a)-[r:`" + relation + "`]->(b)",
        {"subject": subject, "object": obj}
    )
```

存储结构



关键技术点

特性	说明
MERGE	避免重复创建节点，若已存在则匹配
Entity	所有实体统一标记为 Entity 类型
动态关系类型	关系名称直接作为边的类型

1.3 GraphRAG 查询过程

```
def local_search(query: str, G: nx.DiGraph) -> tuple:
    """本地搜索：基于知识图谱回答问题"""
    # 1. LLM解析问题，识别实体和类型
    # 2. 在图中查找多跳路径
    # 3. 基于上下文生成答案
    context = find_multi_hop_paths(G, entities, max_hops=2)
    answer = llm.complete(f"根据以下信息回答问题: {context}\n问题: {query}")
    return context, answer
```

查询流程：

用户问题 → 实体识别 → 图遍历 → 路径查找 → LLM生成答案

↓

"张三和谷歌公司有什么关系?"

↓

识别实体: 张三、谷歌公司

↓

查找路径: 张三 → 父亲 → 李四 → 工作单位 → 谷歌公司

↓

生成答案: 张三通过父亲李四与谷歌公司产生联系

二、普通 RAG 工作原理

2.1 文档处理与索引构建

```
def build_rag_index(documents: list):  
    """构建普通RAG索引"""  
    # 创建 LlamaIndex 文档对象  
    llama_docs = [LlamaDocument(text=doc) for doc in documents]  
  
    # 使用 Ollama Embedding 将文档向量化  
    # 创建向量存储索引  
    rag_index = VectorStoreIndex.from_documents(llama_docs)  
    return rag_index
```

处理流程:

原始文档 → 文本分割 → 向量化 (Embedding) → 向量存储 → 索引构建

↓

"王五是谷歌公司的CEO..." → ["王五是谷歌公司的CEO", "他提倡人工智能技术"]

↓

→ [向量1], [向量2] (768维浮点数)

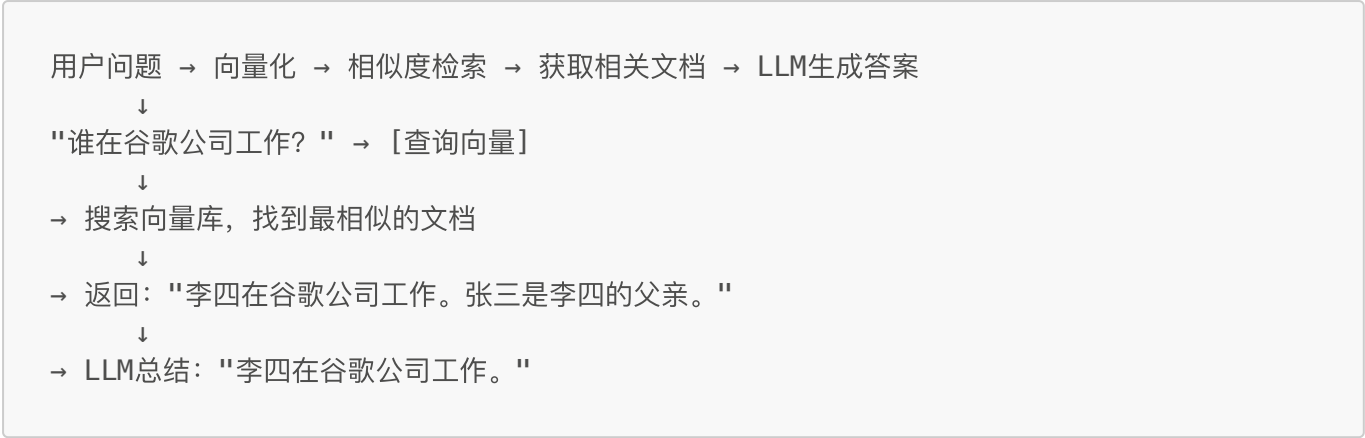
↓

→ 存入内存向量库

2.2 RAG 查询过程

```
def rag_query(query: str) -> str:  
    """普通RAG查询"""  
    query_engine = rag_index.as_query_engine()  
    response = query_engine.query(query)  
    return str(response)
```

查询流程:



三、GraphRAG vs 普通 RAG 对比

3.1 核心差异

维度	GraphRAG	普通 RAG
数据结构	知识图谱（实体+关系）	向量数据库
存储方式	Neo4j（持久化）	内存向量库
查询方式	图遍历 + 关系推理	向量相似度搜索
推理能力	支持多跳关系推理	仅基于文本相似度
可解释性	可展示推理路径	仅展示相关文档
隐式知识	需要显式三元组	可从文本隐式推理
聚合统计	支持图统计查询	不支持

3.2 能力对比示例

问题：张三和谷歌公司有什么关系？

系统	回答
GraphRAG	张三 → 父亲 → 李四 → 工作单位 → 谷歌公司。 因此：张三是通过他的父亲李四与谷歌公司产生联系的。
普通RAG	根据文档"张三是李四的父亲，李四在谷歌公司工作"，可以推断张三与谷歌公司有间接关系。

问题：有多少人在科技公司工作？

系统	回答
GraphRAG	需要显式的"科技公司"类型标记和工作关系
普通RAG	根据文档内容，可以推断李四在谷歌公司（科技公司）工作，因此至少有1人。

3.3 适用场景

场景	推荐方案	原因
多跳关系推理	GraphRAG	显式的关系结构支持路径查找
隐式知识问答	普通RAG	可从完整文本中推理
路径可视化	GraphRAG	天然支持图结构展示
快速原型开发	普通RAG	实现简单，无需图数据库
数据持久化	GraphRAG	Neo4j支持持久化存储

四、项目架构

4.1 文件结构

```
graphrag/
├── backend/
│   └── app.py                # Flask后端API服务
├── frontend/
│   ├── server.js            # Node.js前端服务
│   ├── package.json         # 前端依赖配置
│   └── public/
│       ├── index.html       # 前端页面
│       ├── style.css        # 样式文件
│       └── app.js            # 前端逻辑
├── data/
│   ├── sample_documents.txt  # 示例文档数据
│   └── test_questions.txt    # 测试问题集
├── start_full.sh             # 完整启动脚本
├── stop_full.sh              # 完整停止脚本
├── start_graphrag.sh         # GraphRAG启动脚本
├── stop_graphrag.sh          # GraphRAG停止脚本
└── requirements.txt          # Python依赖配置
```

4.2 API 端点

端点	方法	功能
/api/import	POST	导入文档到知识图谱
/api/import-file	POST	从文件导入文档到知识图谱
/api/query	POST	GraphRAG查询
/api/graph	GET	获取图谱结构数据
/api/status	GET	检查服务状态
/api/rag-import	POST	导入文档到RAG索引
/api/rag-query	POST	普通RAG查询

端点	方法	功能
/api/compare-query	POST	对比查询（同时返回两种结果）

五、使用说明

5.1 启动服务

```
# 启动完整服务 (Neo4j + 后端 + 前端)
./start_full.sh

# 或分别启动
neo4j start
python backend/app.py
node frontend/server.js
```

5.2 访问界面

打开浏览器访问：<http://localhost:3000>

5.3 导入数据

1. 方式一：导入示例数据

```
curl -X POST http://localhost:5001/api/import \
-H "Content-Type: application/json" \
-d '{"documents": [{"content": "张三是李四的父亲，李四在谷歌公司工作。"}]}'
```

2. 方式二：从文件导入

```
curl -X POST http://localhost:5001/api/import-file \
-H "Content-Type: application/json" \
-d '{"file_path": "data/sample_documents.txt"}'
```

5.4 查询示例

```
# GraphRAG查询
curl -X POST http://localhost:5001/api/query \
-H "Content-Type: application/json" \
-d '{"query": "张三和谷歌公司有什么关系?"}'

# 普通RAG查询
curl -X POST http://localhost:5001/api/rag-query \
-H "Content-Type: application/json" \
```

```
-d '{"query": "张三和谷歌公司有什么关系? "'

# 对比查询
curl -X POST http://localhost:5001/api/compare-query \
-H "Content-Type: application/json" \
-d '{"query": "张三和谷歌公司有什么关系? "'
```

5.5 停止服务

```
./stop_full.sh
```

六、技术栈

组件	技术	版本
后端框架	Flask	2.0+
图数据库	Neo4j	5.0+
LLM	Ollama (Qwen2.5)	-
向量模型	Ollama (mxbai-embed-large)	-
向量索引	LlamaIndex	0.10+
图处理	NetworkX	3.0+
前端框架	Node.js + Express	18+

七、关键概念总结

7.1 知识图谱

由**实体**（节点）和**关系**（边）组成的结构化数据表示。

7.2 三元组

知识图谱的基本单元：**（主语，谓语，宾语）**，如**（张三，父亲，李四）**。

7.3 多跳推理

通过多个关系路径进行推理，如：**张三 → 父亲 → 李四 → 工作单位 → 谷歌公司**。

7.4 向量嵌入（Embedding）

将文本转换为数值向量，用于计算文本相似度。

7.5 RAG（Retrieval-Augmented Generation）

先检索相关文档，再基于检索结果生成答案的生成式AI技术。

八、教学建议

8.1 演示顺序

1. **基础查询**：展示两种系统都能回答简单问题
2. **多跳推理**：展示GraphRAG的关系推理优势
3. **隐式知识**：展示普通RAG的文本推理优势
4. **可视化**：展示知识图谱的结构

8.2 推荐测试问题

```
# 简单问题（两者都能答）
"苹果公司位于哪里？"
"谁提倡人工智能技术？"

# 多跳推理（GraphRAG更强）
"张三和谷歌公司有什么关系？"
"王五和苹果公司有关系吗？"

# 隐式推理（普通RAG更强）
"有多少人在科技公司工作？"
"根据提供的信息，总结张三的背景。"
```

九、常见问题

Q1：为什么GraphRAG无法回答某些问题？

A：GraphRAG需要**显式的三元组**才能回答问题。如果知识图谱中缺少相关关系，就无法回答。

Q2：普通RAG的优势是什么？

A：普通RAG可以从**完整文本**中进行隐式推理，不需要预先定义关系结构。

Q3：如何选择合适的方案？

A：根据具体场景选择：

- 需要关系推理和可视化 → GraphRAG
- 需要快速原型和隐式推理 → 普通RAG
- 生产环境建议 → 两者结合

文档版本: v1.0

生成日期: 2026年5月

项目地址: ./graphrag